École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Rapport de stage de 2ème année

« Conception et développement d'une plateforme full-stack dédiée à l'artisanat d'art »

Étudiante : **Manal Derghal**
Tuteur académique : **Julien Forest**

Entreprise : **RUHENNA** – Rue Edmond Boris, 78130 Les Mureaux

Maître de stage : **Sumeyra Solak**

Dates : **Du 19 janvier au 13 mars 2026**

Résumé

Ce rapport présente le travail réalisé au sein de l'entreprise **RUHENNA**, une activité artisanale portée par **Sumeyra Solak**, dans le cadre d'un stage de 2ème année. L'objectif du projet était de concevoir et développer une plateforme web *full-stack* permettant de renforcer l'image de marque, de valoriser les créations autour de l'art du henné et de simplifier la gestion opérationnelle (catalogue de produits, prise de rendez-vous, blog et formulaire de contact), tout en offrant un **panneau d'administration** pour centraliser l'édition du contenu et le suivi des demandes.

La solution s'appuie sur une architecture moderne basée sur **React** et **Next.js** (App Router), **Prisma** comme ORM, et une base de données **PostgreSQL** hébergée sur **Neon** pour faciliter le déploiement sur **Vercel**. La gestion des médias s'appuie sur **Cloudinary** afin d'optimiser le stockage et la diffusion d'images. Une attention particulière a été portée à l'expérience utilisateur (interfaces publiques et administration), à la structuration des données (modèle relationnel, contraintes d'intégrité) et à la maintenabilité (typage TypeScript, validation des entrées, séparation des responsabilités).

Au-delà des aspects techniques, ce stage a constitué une montée en compétences progressive sur React/Next.js et Prisma, depuis un niveau initial limité, à travers un apprentissage guidé par la documentation et des ressources en ligne, puis par la confrontation à des problèmes réels (routage, interaction front/back, disponibilité de créneaux, gestion d'images, déploiement).

Mots-clés

Mots-clés : Next.js, React, Prisma, PostgreSQL, réservation, back-office, Cloudinary, Zod, TipTap, @dnd-kit.

Table des matières

Résumé **i**

1 Introduction : environnement de travail et stage **1**
1.1 Présentation de l'environnement 1
1.2 Sujet et objectifs du stage 1
1.3 Outils et moyens utilisés 1
1.4 Annonce du plan 1

2 Contexte et préliminaires **2**
2.1 Environnement 2
2.1.1 Entreprise et activité 2
2.1.2 Organisation et contraintes 2
2.2 Problématique : contexte et objectifs 2
2.3 Spécifications du stage (cahier des charges) 2
2.3.1 Exigences fonctionnelles 2
2.3.2 Exigences non-fonctionnelles 3
2.4 Méthodologie et outils mobilisés 3
2.4.1 Approche de réalisation 3
2.4.2 Montée en compétences 3
2.4.3 Outils et briques techniques du projet 3
2.5 Choix technologiques : justification et alternatives 4
2.5.1 Pourquoi React et Next.js ? 4
2.5.2 Pourquoi Prisma et PostgreSQL ? 4
2.5.3 Comparatif : autres technologies possibles (front, back, base de données) 4
2.6 Schémas pédagogiques : TypeScript et Prisma 5
2.6.1 Comprendre TypeScript : typage statique et compilation 5
2.6.2 Comprendre Prisma : schéma, migrations et client 6
2.7 Architecture logicielle à étendre 6

3 Solution implémentée et contributions **7**
3.1 Étude des solutions potentielles (avant choix final) 7
3.1.1 Option 1 : site vitrine statique (CMS ou générateur) 7
3.1.2 Option 2 : architecture séparée Front + Back 7
3.1.3 Option retenue : Next.js full-stack + Prisma + PostgreSQL 7
3.2 Solution implémentée : architecture et composants 7
3.2.1 Vue d'ensemble technique 7
3.2.2 Structure du dépôt et conventions Next.js 8

3.2.3 Routage : pages publiques, administration et API . . . 8
3.3 Base de données : schéma, entités, relations . . . 8
3.3.1 Rôle de Prisma (ORM) . . . 8
3.3.2 ERD (diagramme entité-association) . . . 8
3.3.3 Modèle de données . . . 9
3.4 Fonctionnalités du front public . . . 9
3.4.1 Page d'accueil et identité de marque . . . 9
3.4.2 Catalogue (boutique) : logique de lecture . . . 10
3.4.3 Galerie : consultation . . . 10
3.4.4 Blog : contenu riche et SEO . . . 10
3.5 Système de réservation : calcul de créneaux et création d'un rendez-vous . . . 10
3.5.1 Parcours utilisateur (5 étapes) . . . 10
3.5.2 Calcul des créneaux disponibles (API) . . . 10
3.5.3 Création d'une demande de rendez-vous . . . 10
3.6 Formulaire de contact : validation et stockage . . . 11
3.6.1 API de contact . . . 11
3.7 Communication entre l'interface React et l'API . . . 11
3.7.1 Validation des formulaires avec Zod . . . 11
3.7.2 Éditeur de contenu riche (TipTap) pour le blog . . . 11
3.7.3 Glisser-déposer (@dnd-kit) pour réordonner . . . 12
3.8 Administration : gestion CRUD, tri, statuts . . . 12
3.8.1 Navigation et organisation . . . 12
3.8.2 Produits et catégories . . . 12
3.8.3 Galerie et bannière : réorganisation via drag & drop . . . 12
3.8.4 Rendez-vous : statuts et édition . . . 12
3.8.5 Messages : inbox et archives . . . 12
3.8.6 Autonomie de gestion pour l'entreprise . . . 12
3.9 Gestion des images : upload Cloudinary . . . 13
3.9.1 Raison d'être . . . 13
3.9.2 Endpoint d'upload . . . 13
3.10 Sécurité : protection de l'administration et en-têtes HTTP . . . 13
3.10.1 Protection Basic Auth et en-têtes de sécurité . . . 13
3.11 Déploiement : Vercel + Neon + domaine . . . 13

4 Conclusion et perspectives **14**
4.1 Conclusion . . . 14
4.2 Difficultés et leçons apprises . . . 14
4.2.1 Difficulté principale : apprendre React/Next.js et Prisma en situation réelle . . . 14
4.2.2 Leçons apprises . . . 14
4.3 Perspectives et évolutions possibles . . . 15
4.4 Apports pour l'établissement . . . 15

Bibliographie **16**

Glossaire **17**

A DD&RS — Développement durable et responsabilité sociale **19**
A.1 Impacts environnementaux et sociétaux (DD&RS) 19
A.1.1 Impacts liés à l'activité de l'entreprise 19
A.1.2 Impacts liés au numérique (plateforme web) 19
A.1.3 Impacts liés à l'équipe / au contexte de travail 19
A.2 Actions de réduction des impacts 19
A.2.1 Actions mises en œuvre 19
A.2.2 Suppression des images côté Cloudinary 20

B Structure de l'entreprise **21**
B.1 Organisation générale 21
B.2 Organigramme simplifié 21
B.3 Conséquences sur le produit 21

C Détails techniques **22**
C.1 Versionnement et stack observée 22
C.2 Variables d'environnement 22
C.3 Structure des dossiers Next.js (public, admin, API) 22
C.4 Inventaire des principales routes API 23
C.5 Accès base de données (Prisma Client) 24
C.6 Configuration Cloudinary 24
C.7 Optimisation des images côté Next.js 25
C.8 Dépôt GitHub (lien du code) 25

Chapitre 1

Introduction : environnement de travail et stage

1.1 Présentation de l'environnement

Ce stage de 2ème année a été réalisé au sein de **RUHENNA**, une activité artisanale portée par **Sumeyra Solak** et centrée sur l'art du henné (prestations pour événements et produits associés). La structure étant de petite taille, l'enjeu principal était de livrer une solution **simple à utiliser** et **administrable** au quotidien.

1.2 Sujet et objectifs du stage

Le sujet du stage était de participer à la conception et au développement d'une plateforme web *full-stack* permettant :

— de valoriser la marque et les réalisations (galerie, blog) ;
— de rendre l'offre lisible (catalogue) ;
— de structurer les demandes (contact et réservation) ;
— de permettre une gestion autonome via un espace d'administration.

1.3 Outils et moyens utilisés

Le projet s'appuie sur une stack web courante : **React/Next.js** (App Router), **Prisma** et **PostgreSQL** (hébergée sur **Neon**), **Tailwind CSS**, **Cloudinary** pour les images, et un déploiement sur **Vercel**.

1.4 Annonce du plan

Le rapport présente d'abord le contexte et les choix (contraintes, outils, méthode, alternatives), puis détaille la solution mise en place, les contributions et les résultats observables (exemples de parcours, gestion admin, mise en production). Il se termine par une synthèse et des perspectives, avec des annexes techniques.

Chapitre 2

Contexte et préliminaires

2.1 Environnement

2.1.1 Entreprise et activité

RUHENNA est une activité artisanale portée par **Sumeyra Solak**. Elle propose des prestations au henné (notamment pour des événements) et des produits associés (cônes, accessoires, kits). Le henné est une teinture d'origine végétale (*Lawsonia inermis*) qui permet de réaliser des motifs temporaires sur la peau, avec une dimension culturelle importante lors de certaines célébrations.

2.1.2 Organisation et contraintes

La structure ne disposant pas d'équipe informatique, les échanges ont été directs : Sumeyra porte la vision métier (offre, contenus, règles de réservation) et le stage vise à livrer un outil administrable. Le besoin a été identifié de manière concrète à la suite d'échanges initiés dans un contexte réel (Sumeyra a réalisé le henné lors de mon mariage), puis formalisé en fonctionnalités et règles simples.

2.2 Problématique : contexte et objectifs

Avant la plateforme, la présence en ligne et la gestion des demandes pouvaient être dispersées (réseaux sociaux, messages, échanges manuels). L'objectif a donc été de concevoir un site qui valorise l'activité (galerie, blog, catalogue) tout en structurant la prise de contact et la réservation, avec un back-office permettant une mise à jour autonome.

2.3 Spécifications du stage (cahier des charges)

2.3.1 Exigences fonctionnelles

Le cahier des charges fonctionnel a été construit à partir des besoins concrets de l'activité :

— **Catalogue (boutique)** : afficher les produits sous forme de *catalogue*, avec une page détail par produit.
— **Catégories** : proposer des catégories ordonnées afin de guider la navigation.
— **Galerie** : exposer des réalisations, avec possibilité de mise en avant.
— **Bannière (hero)** : gérer un carrousel d'images sur la page d'accueil.
— **Réservation** : permettre la sélection d'un service, d'une date et d'un créneau disponible, puis l'envoi d'une demande.

— **Contact** : formulaire structuré, stockage du message en base, consultation côté admin.
— **Blog** : création d'articles, publication/dépublication, lecture publique.
— **Administration** : CRUD complet des entités (produits, services, images, articles, messages, rendez-vous).

2.3.2 Exigences non-fonctionnelles

— **Maintenabilité** : code lisible, typé, organisé, et facilement extensible.
— **Performance et UX** : mise en cache implicite, chargements fluides, formulaires guidés.
— **SEO** : pages indexables, métadonnées, données structurées (LocalBusiness).
— **Sécurité** : protection de l'admin, validation des entrées, en-têtes de sécurité.
— **Gestion des médias** : upload sécurisé, optimisation des images.

2.4 Méthodologie et outils mobilisés

2.4.1 Approche de réalisation

L'activité RUHENNA étant de petite taille, il n'existait pas de méthodologie projet formalisée (Scrum complet, etc.). Le travail a néanmoins été structuré par une logique itérative :

1. **Cadrage** : clarification des fonctionnalités prioritaires (vitrine, galerie, réservation, admin).
2. **Modélisation** : construction du schéma de données (Prisma/PostgreSQL).
3. **Implémentation verticale** : livraison d'une fonctionnalité complète (UI + API + DB), puis stabilisation.
4. **Validation** : tests manuels et retours d'usage (cohérence des formulaires, lisibilité admin).

2.4.2 Montée en compétences

Un point important du stage est le niveau initial : il y avait **peu de maîtrise préalable** de React/Next.js et de Prisma. L'apprentissage s'est appuyé sur :
— la documentation officielle ;
— des tutoriels en ligne ;
— l'expérimentation directe.
Cette approche a été déterminante : l'apprentissage a progressé au rythme des problèmes réels.

2.4.3 Outils et briques techniques du projet

Les principaux outils observés dans le code sont :
— **Next.js / React** pour l'interface et le routage *App Router* ;
— **TypeScript** pour le typage et la fiabilité ;
— **Prisma** comme ORM ; **PostgreSQL** comme SGBD ;
— **Neon** pour héberger PostgreSQL de façon compatible avec le déploiement sur **Vercel** ;
— **Tailwind CSS** pour le style et la cohérence graphique ;
— **Cloudinary** pour le stockage d'images ;
— **Zod** pour la validation des entrées côté API ;
— **TipTap** pour l'édition des articles ;
— **@dnd-kit** pour le glisser-déposer.

2.5 Choix technologiques : justification et alternatives

2.5.1 Pourquoi React et Next.js ?

Le choix de React s'explique par la nécessité de construire deux expériences riches : (i) un front public (galerie, boutique, blog, réservation), et (ii) un back-office interactif (formulaires, modales, tri, filtres). Next.js a été retenu pour :

— son **routage par convention** (`src/app/...`) ;
— ses **Route Handlers** permettant d'intégrer l'API sans serveur séparé ;
— le rendu hybride (Server Components / Client Components) utile pour la performance et le SEO ;
— l'intégration naturelle avec Vercel pour le déploiement.

Ces choix se fondent notamment sur la documentation officielle Next.js [1].

2.5.2 Pourquoi Prisma et PostgreSQL ?

PostgreSQL apporte un modèle relationnel robuste pour des entités structurées (produits, rendez-vous, disponibilités, messages, articles). Prisma complète cet avantage en fournissant :

— un **schéma unique** (`schema.prisma`) ;
— des **types TypeScript** générés ;
— des migrations et une couche d'accès cohérente, limitant les erreurs de requêtes.

Cette approche correspond aux bonnes pratiques décrites par Prisma [2] : décrire le modèle de données dans un schéma central, versionner les évolutions via des migrations, puis utiliser le client généré (typé) pour effectuer des opérations CRUD de manière cohérente avec le schéma.

2.5.3 Comparatif : autres technologies possibles (front, back, base de données)

Afin de justifier le choix final, un comparatif synthétique d'alternatives réalistes est proposé. L'objectif n'est pas de « prouver » qu'une technologie est meilleure qu'une autre, mais d'expliquer un choix orienté **simplicité** : RUHENNA est un **petit site** (peu de données, pas de volumétrie massive) et la priorité était de livrer une solution stable et facile à maintenir.

TABLE 2.1 – Comparatif d'alternatives (Front / Back / Base de données) — sources : documentations officielles [3, 1, 4, 2, 5].

Couche	Option	Avantages	Limites
Frontend	Next. js (React)	Bon SEO (SSR/SSG), routing par dossiers, un seul projet pour pages + API.	Courbe d'apprentissage (Server/Client), conventions spécifiques.
	Nuxt (Vue)	SSR/SSG intégrés, approche cohérente, Vue plus simple par certains profils.	Stack différente ; moins alignée avec l'écosystème React.
	Angular	Cadre complet, architecture imposée.	Plus lourd pour un petit site ; complexité initiale élevée.
Backend	Route Handlers Next. js	API intégrée, un seul dépôt, mise en production simple.	À structurer proprement si le projet grossit (services, tests).
	Express / NestJS	Séparation claire, patterns backend avancés, tests faciles.	Infrastructure en plus, coût de maintenance plus élevé.
	Django / FastAPI	Productivité Python, écosystème riche.	Double stack (Python + JS), intégration front plus lourde.
Base de données	PostgreSQL	Contraintes, transactions, relations, robustesse.	Modèle relationnel à concevoir correctement.
	MySQL	Mature, performant, populaire.	Moins de fonctionnalités avancées, choix moins naturel selon l'hôte.
	MongoDB (NoSQL)	Flexibilité totale du schéma.	Moins adapté aux contraintes fortes (RDV, règles métier strictes).

2.6 Schémas pédagogiques : TypeScript et Prisma

2.6.1 Comprendre TypeScript : typage statique et compilation

TypeScript apporte une vérification de types avant l'exécution. Les types sont **effacés** après compilation : l'exécution se fait en JavaScript. Le schéma de la Figure 2.1 est inspiré du fonctionnement décrit dans la documentation TypeScript [6].

FIGURE 2.1 – Schéma simplifié du fonctionnement TypeScript : vérification à la compilation, exécution en JavaScript.

2.6.2 Comprendre Prisma : schéma, migrations et client

Prisma s'articule autour d'un schéma déclaratif (`schema.prisma`) décrivant les modèles. À partir de ce schéma, on obtient :

— des **migrations** qui matérialisent les tables/contraintes dans PostgreSQL ;

— un **client Prisma** typé, utilisé dans le code Next.js.

La Figure 2.2 synthétise le cycle recommandé dans la documentation Prisma [2].

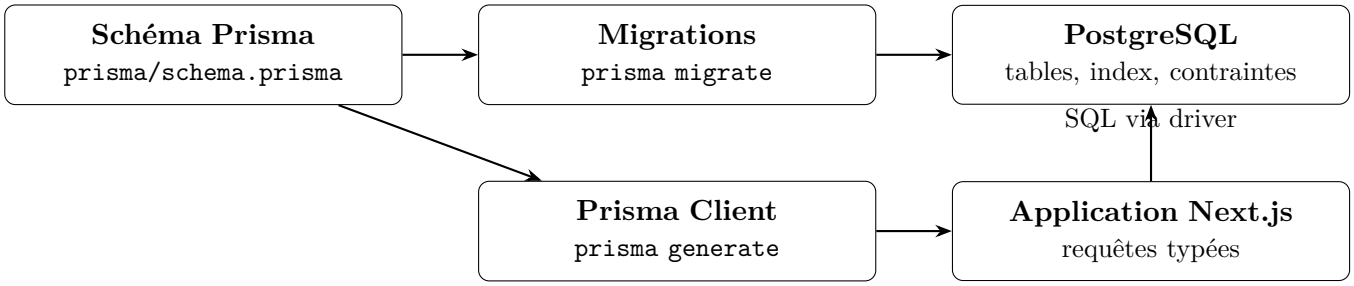

FIGURE 2.2 – Cycle Prisma : du schéma aux migrations et au client typé.

2.7 Architecture logicielle à étendre

Le projet a été développé **from scratch** : il n'existait pas d'architecture applicative préalable au stage. En revanche, le choix de Next.js impose une architecture par conventions (dossier `app/`, layouts, routes API) qui sert de cadre structurel et garantit une cohérence globale. Le chapitre 3 détaillera cette architecture concrète à travers la structure du dépôt et les principales routes/pages.

Chapitre 3

Solution implémentée et contributions

3.1 Étude des solutions potentielles (avant choix final)

Avant de stabiliser la solution, plusieurs options ont été évaluées, en cohérence avec les contraintes du projet : rapidité de développement, possibilité d'évolution (vers e-commerce), simplicité de maintenance et autonomie de l'artisane.

3.1.1 Option 1 : site vitrine statique (CMS ou générateur)

Une première approche consistait à produire un site principalement statique (WordPress, Webflow, ou générateur statique). Cette option présente des avantages (mise en ligne rapide, administration "contenu"), mais atteint rapidement ses limites dès qu'on introduit :

— des règles métier (créneaux de rendez-vous, disponibilité par service) ;
— un back-office sur mesure (tri d'images, gestion des statuts) ;
— une base de données relationnelle cohérente.

Elle a donc été jugée insuffisante pour un besoin évolutif.

3.1.2 Option 2 : architecture séparée Front + Back

Une seconde approche envisageait un front (React) et un back séparé (Express/NestJS ou Django/FastAPI). Cette option est robuste, mais elle augmente le coût d'infrastructure et la charge de maintenance (deux projets, deux déploiements, plus de configuration). Pour RUHENNA, une solution monolithique Next.js est apparue plus adaptée.

3.1.3 Option retenue : Next.js full-stack + Prisma + PostgreSQL

La solution finale combine l'interface (front public et back-office) et l'API dans une seule application Next.js, avec Prisma pour le modèle de données et PostgreSQL (Neon) pour le stockage, afin d'obtenir une base saine et une mise en production simple sur Vercel.

3.2 Solution implémentée : architecture et composants

3.2.1 Vue d'ensemble technique

La plateforme est structurée autour de quatre blocs :

— **Front public** : pages accessibles aux visiteurs (vitrine, galerie, blog, boutique, réservation, contact) ;
— **Back-office** : pages `/admin` pour gérer le contenu et les demandes ;
— **API** : endpoints REST via les Route Handlers Next.js (`/api/...`) ;
— **Données et médias** : PostgreSQL (Neon) via Prisma, et Cloudinary pour les images.

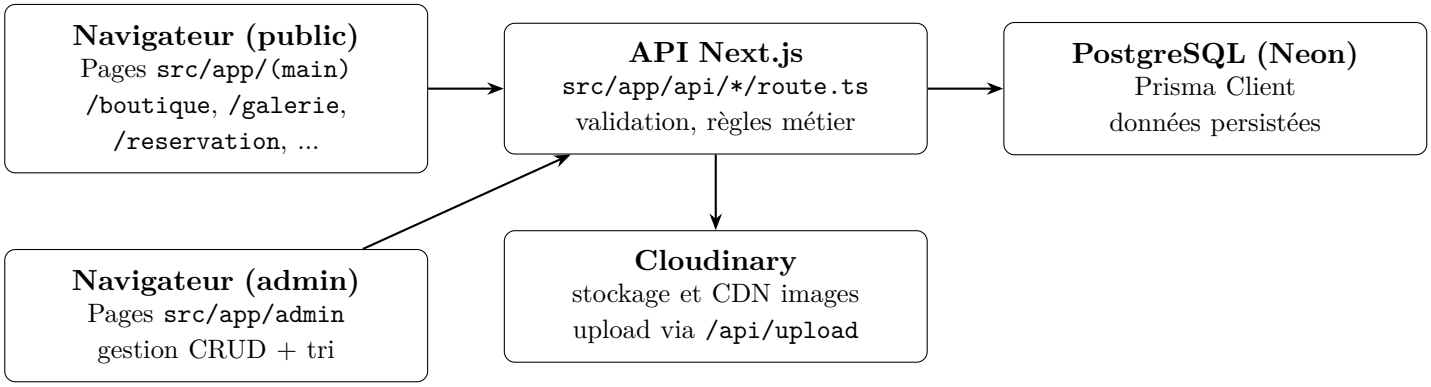

FIGURE 3.1 – Architecture applicative : Next.js centralise UI et API, Prisma relie l'application à PostgreSQL, Cloudinary gère les médias.

3.2.2 Structure du dépôt et conventions Next.js

Le code suit l'architecture par conventions de Next.js (App Router). Les dossiers principaux sont :
— `src/app/` : pages, layouts, et endpoints API ;
— `src/components/` : composants UI réutilisables (header/footer, éditeur, uploaders, etc.) ;
— `src/lib/` : accès DB, validation, configuration Cloudinary, utilitaires ;
— `prisma/` : schéma Prisma, migrations, seed et ERD.

3.2.3 Routage : pages publiques, administration et API

Trois familles de routes coexistent :

1. **Routes publiques** : `src/app/(main)/...` (ex. `/boutique`, `/blog`).
2. **Routes admin** : `src/app/admin/...` (ex. `/admin/produits`, `/admin/blog`).
3. **Routes API** : `src/app/api/.../route.ts` (ex. `/api/contact`, `/api/rendez-vous`).

3.3 Base de données : schéma, entités, relations

3.3.1 Rôle de Prisma (ORM)

Prisma agit comme une couche intermédiaire entre l'application et PostgreSQL. Les modèles sont définis dans `prisma/schema.prisma` et Prisma génère un client typé, utilisé dans les pages serveur et les routes API.

3.3.2 ERD (diagramme entité-association)

Le diagramme ERD a été généré à partir du schéma Prisma. Il permet d'illustrer les principales entités (*produits, catégories, services, rendez-vous, blog, messages, images*) et les liens entre elles.

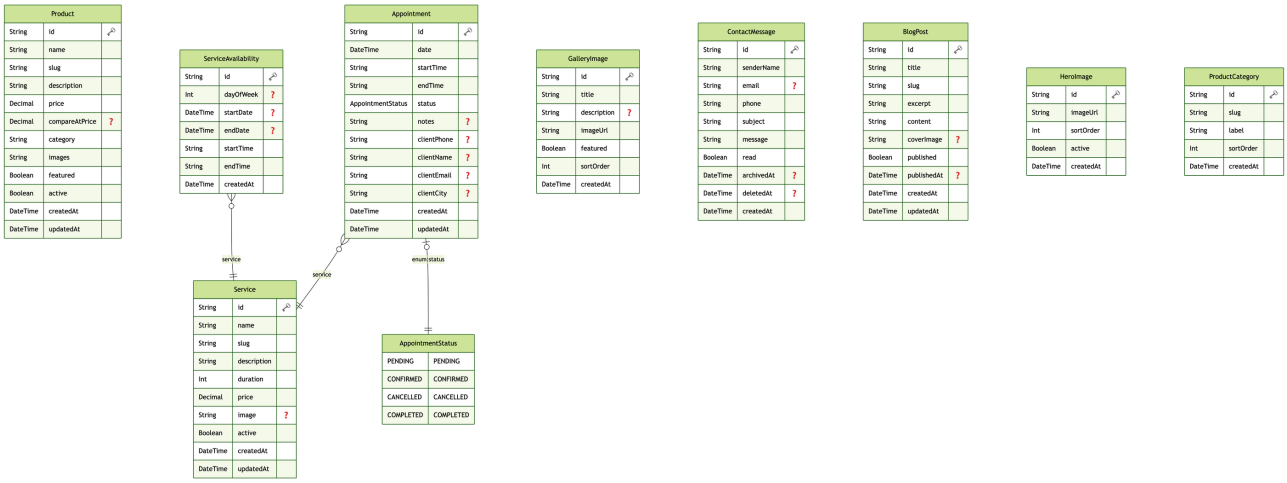

FIGURE 3.2 – ERD de la base de données RUHENNA (généré depuis `prisma/schema.prisma`).

3.3.3 Modèle de données

Le modèle de données s'organise autour d'entités qui correspondent directement aux besoins du site (catalogue, réservation, contenu, médias, messages) :

— **Produits** (`Product`) : nom, description, prix, catégorie, liste d'URLs d'images (Cloudinary), statut actif/mis en avant.
— **Catégories** (`ProductCategory`) : liste ordonnée (champ `sortOrder`) pour maîtriser l'affichage de la boutique.
— **Services** (`Service`) : durée et prix, associés à des règles de disponibilité (`ServiceAvailability`).
— **Rendez-vous** (`Appointment`) : date, heure de début/fin, statut (en attente, confirmé, annulé, terminé).
— **Blog** (`BlogPost`) : titre, slug, extrait, contenu HTML, image de couverture, publication.
— **Messages** (`ContactMessage`) : informations du demandeur et contenu, avec gestion "lu / archivé / supprimé".
— **Médias** : images de galerie et de bannière (`GalleryImage`, `HeroImage`) avec ordre d'affichage.

Pour éviter deux rendez-vous au même horaire, une contrainte d'unicité impose qu'à une date donnée, un même `startTime` ne puisse pas être réservé deux fois. Lors de la création d'un rendez-vous, l'application vérifie aussi l'absence de **chevauchement** horaire avec les rendez-vous déjà enregistrés.

3.4 Fonctionnalités du front public

3.4.1 Page d'accueil et identité de marque

La page d'accueil combine :

— un **hero** avec carrousel d'images (`HeroImage`) ;
— une mise en avant des réalisations (`GalleryImage.featured`) ;
— des appels à action (galerie et réservation).

Le choix d'une page d'accueil dynamique permet de publier rapidement de nouvelles réalisations, sans redeployer manuellement des assets.

3.4.2 Catalogue (boutique) : logique de lecture

La boutique est conçue comme un **catalogue** : les produits sont consultables, filtrables par catégories, et la prise de commande passe par le contact (pas de paiement intégré à ce stade). Cette approche correspond à une phase où l'offre évolue et où l'artisane souhaite conserver un lien direct avec la clientèle.

3.4.3 Galerie : consultation

La galerie propose une grille responsive afin de mettre en valeur les réalisations, sans ajouter de complexité inutile dans le parcours utilisateur.

3.4.4 Blog : contenu riche et SEO

Les articles sont stockés en base (table `BlogPost`) et publiés/dépubliés via l'administration.

3.5 Système de réservation : calcul de créneaux et création d'un rendez-vous

3.5.1 Parcours utilisateur (5 étapes)

Le formulaire de réservation est conçu en plusieurs étapes (service, date, horaire, informations, récapitulatif) afin de réduire les erreurs et de guider la saisie.

3.5.2 Calcul des créneaux disponibles (API)

L'endpoint `/api/rendez-vous/slots` calcule les créneaux à partir :
— de la durée du service ;
— des règles `ServiceAvailability` ;
— des rendez-vous existants non annulés (détection de chevauchement).

3.5.3 Création d'une demande de rendez-vous

Une fois un créneau choisi, `/api/rendez-vous` crée un enregistrement `Appointment` en statut `PENDING`. L'API vérifie :
— la validité du payload (**Zod**) ;
— l'existence du service ;
— la non-disponibilité du créneau (conflit).
Le navigateur envoie un JSON minimal à l'API, avec notamment :
— `serviceId` : *"ck..."*
— `date` : *"AAAA-MM-JJ"*
— `startTime` : *"11 :00"*
— `name/email/phone/city` : informations de contact
Si tout est valide, l'API crée un rendez-vous **en attente** et renvoie un identifiant de suivi. Si le créneau est pris entre-temps, elle renvoie un message "créneau indisponible".

3.6 Formulaire de contact : validation et stockage

3.6.1 API de contact

La route `/api/contact` illustre l'utilisation de Zod pour valider l'entrée et de Prisma pour persister le message. Un message "Demande de devis" contient des champs simples (nom, email, sujet, message). Si un champ est mal formé (email invalide, message trop court), la réponse contient une liste d'erreurs *par champ*, ce qui permet d'afficher une aide claire à l'utilisateur.

3.7 Communication entre l'interface React et l'API

Les formulaires (contact, réservation, administration) sont des composants React côté navigateur (*Client Components*). Ils envoient des requêtes HTTP à l'API (`/api/...`), qui valide les données (Zod), puis appelle Prisma pour écrire/lire dans PostgreSQL.

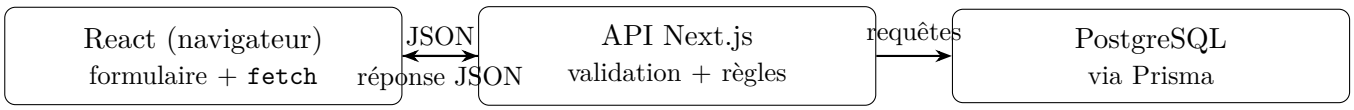

FIGURE 3.3 – Flux simplifié : UI React → API → base de données.

Le formulaire "Contact" construit un objet JSON avec les champs saisis, puis l'envoie à `/api/contact`. L'API répond soit :

— **Succès** : `{ success: true }` + message de confirmation ;
— **Erreurs** : `{ success: false, errors: ... }` pour afficher les champs à corriger.

3.7.1 Validation des formulaires avec Zod

Zod sert à décrire ce qui est attendu (types, formats, longueurs) et à refuser des données incohérentes [7]. Les erreurs sont **structurées** et donc faciles à afficher côté interface.

Dans le cas d'une réservation, si l'objet reçu ne respecte pas la structure attendue (date qui n'est pas au format `AAAA-MM-JJ`, email invalide, nom trop court), la validation échoue et renvoie des erreurs associées aux champs, comme :

— `date` : "format de date invalide" ;
— `email` : "adresse email invalide" ;
— `name` : "au moins 2 caractères".

De la même façon, si un champ obligatoire manque (par exemple `serviceId`) ou si un champ n'a pas le bon type, la requête est rejetée avant toute écriture en base.

3.7.2 Éditeur de contenu riche (TipTap) pour le blog

Le back-office propose un éditeur riche basé sur TipTap [8]. Concrètement :

— l'artisane écrit avec des outils (titres, listes, gras, images, liens) ;
— le contenu est enregistré en base sous forme de **HTML** ;
— la page publique "Article" réaffiche ce HTML (contenu maîtrisé car produit via l'admin).

Cette approche évite d'imposer du HTML à l'utilisatrice, tout en gardant une mise en forme cohérente.

3.7.3 Glisser-déposer (@dnd-kit) pour réordonner

Pour les catégories, la galerie ou la bannière, l'ordre d'affichage est important. La bibliothèque `@dnd-kit` [9] permet :

— de déplacer un élément dans une liste (drag & drop) ;
— de recalculer l'ordre côté UI ;
— d'envoyer au serveur la liste ordonnée des identifiants (endpoint *reorder*) ;
— de persister l'ordre en base (mise à jour du champ `sortOrder`).

Si l'image "Mariage" doit apparaître en premier, on la glisse en haut ; l'admin envoie alors les `ids` dans le nouvel ordre.

3.8 Administration : gestion CRUD, tri, statuts

3.8.1 Navigation et organisation

L'espace d'administration (`/admin`) propose un menu donnant accès aux modules : bannière, produits, catégories, services, galerie, rendez-vous, messages, blog.

3.8.2 Produits et catégories

Les produits sont gérés via `/api/admin/products`. Les catégories (`ProductCategory`) sont ordonnées et peuvent être réorganisées en drag & drop.

3.8.3 Galerie et bannière : réorganisation via drag & drop

La galerie et la bannière implémentent des routes de **reorder** afin de persister l'ordre côté base (champ `sortOrder`). Le glisser-déposer est réalisé via `@dnd-kit`.

3.8.4 Rendez-vous : statuts et édition

L'administration des rendez-vous permet de confirmer/annuler, d'éditer les informations client, et de créer manuellement un rendez-vous confirmé. L'API `/api/admin/rendez-vous` centralise ces opérations.

3.8.5 Messages : inbox et archives

Les messages issus du formulaire de contact sont consultables en "boîte de réception", marqués lus/non lus, archivés, puis supprimés.

3.8.6 Autonomie de gestion pour l'entreprise

L'objectif du back-office est que Sumeyra puisse gérer le site **sans dépendre** d'une intervention technique pour les mises à jour courantes. Concrètement, l'administration permet :

— d'ajouter un nouveau produit, de changer un prix, de désactiver un produit indisponible ;
— d'organiser les catégories (ordre d'affichage) pour rendre la boutique plus lisible ;
— de publier un article de blog (mise en page via éditeur riche) pour communiquer sur une prestation ou une période de forte demande ;
— d'ajouter de nouvelles images de galerie ou de bannière et d'en ajuster l'ordre d'affichage ;
— de consulter les messages et les demandes de rendez-vous, puis de les confirmer/annuler et d'archiver les échanges.

Cette logique vise une utilisation quotidienne simple : mise à jour du contenu, suivi des demandes, et maintien d'une vitrine cohérente.

3.9 Gestion des images : upload Cloudinary

3.9.1 Raison d'être

Les images sont centrales (galerie, produits, blog). Stocker ces fichiers dans le dépôt ou sur le serveur applicatif serait peu robuste. Cloudinary fournit un stockage et un CDN, avec des URLs optimisées.

3.9.2 Endpoint d'upload

L'endpoint `/api/upload` :
— reçoit des fichiers en `multipart/form-data`,
— contrôle type et taille,
— upload dans un dossier Cloudinary,
— retourne des URLs sécurisées.

Lorsque l'admin ajoute des images à un produit, l'API renvoie une liste d'URLs Cloudinary (`https://res.cloudinary.com/...`). Ces URLs sont ensuite stockées dans la base, ce qui permet à l'application d'afficher les images sans gérer de stockage de fichiers "maison".

3.10 Sécurité : protection de l'administration et en-têtes HTTP

3.10.1 Protection Basic Auth et en-têtes de sécurité

La logique de protection est factorisée dans `src/proxy.ts`. Elle applique :
— une authentification **HTTP Basic** sur `/admin` et `/api/admin` via des variables d'environnement (`ADMIN_USER`, `ADMIN_PASS`) ;
— des en-têtes HTTP (anti-clickjacking, nosniff, HSTS, etc.).

Lorsqu'un utilisateur tente d'accéder à `/admin`, le navigateur demande des identifiants. Sans ces identifiants, la page est refusée (`401`). Avec les bons identifiants, la navigation fonctionne normalement et des en-têtes de sécurité sont ajoutés à la réponse.

3.11 Déploiement : Vercel + Neon + domaine

Le déploiement cible s'appuie sur **Vercel** pour sa simplicité (déploiements automatiques, prévisualisations). La base PostgreSQL est provisionnée sur **Neon** et utilisée via `DATABASE_URL` [5]. Les médias sont externalisés sur Cloudinary [10]. Un **nom de domaine personnalisé** est prévu, mais n'est pas encore finalisé à ce stade.

Chapitre 4

Conclusion et perspectives

4.1 Conclusion

Ce stage a permis de livrer une plateforme cohérente et exploitable pour RUHENNA : un espace public orienté valorisation visuelle et prise de contact, et un back-office permettant à l'artisane de gérer son contenu et ses demandes de manière autonome. Le projet répond ainsi à une problématique réelle d'activité indépendante : **structurer** l'offre, **centraliser** les demandes, et **renforcer** la crédibilité en ligne.

4.2 Difficultés et leçons apprises

4.2.1 Difficulté principale : apprendre React/Next.js et Prisma en situation réelle

La difficulté majeure n'a pas été un obstacle isolé, mais une montée en complexité progressive liée au fait de **ne pas "savoir coder" initialement dans cet écosystème**. En pratique, cela s'est traduit par :

- une compréhension progressive du modèle React (composants, état, effets, séparation UI/logique) ;
- l'assimilation du paradigme Next.js App Router (layouts, pages, Server Components vs Client Components) ;
- l'apprentissage de Prisma comme ORM (schéma, migrations, client typé, contraintes) ;
- la mise en relation front/back : concevoir une API, valider les entrées, gérer les statuts, et persister en base.

4.2.2 Leçons apprises

Les principales leçons tirées de ce stage sont les suivantes :

- **Anticiper la conception est essentiel** : investir du temps dans la modélisation (schéma de données, architecture) permet de réduire significativement la dette technique et les corrections ultérieures.
- **S'appuyer sur des outils structurants** : l'utilisation de technologies comme le typage statique (TypeScript), les validations d'entrées et les ORM contribue à améliorer la robustesse, la cohérence et la maintenabilité du code.
- **L'importance de la communication entre composants** : une bonne synchronisation entre les différentes couches (front-end, back-end, base de données) est indispensable pour éviter les incompréhensions et les incohérences fonctionnelles.

— **Itérer et tester régulièrement** : valider fréquemment les fonctionnalités permet de détecter rapidement les erreurs et d'ajuster les choix techniques en continu.

4.3 Perspectives et évolutions possibles

Le projet est conçu pour évoluer. Les améliorations les plus naturelles sont :
— **E-commerce complet** : panier, commande, gestion stock, frais de livraison.
— **Paiement en ligne** : intégration Stripe (paiements, acomptes pour événements).
— **Comptes utilisateurs** : suivi de commandes, historique de rendez-vous.
— **Emails transactionnels et marketing** : confirmation et rappel de rendez-vous, campagnes promotionnelles.

4.4 Apports pour l'établissement

Ce stage a permis de mobiliser et d'illustrer des notions enseignées au cours de la formation, notamment en développement web avec HTML et JavaScript. Il met en évidence l'importance de la structuration des données, de la logique applicative et de la cohérence entre les différentes parties d'un projet web. Il constitue également un exemple concret de mise en application des bonnes pratiques abordées en formation, telles que l'organisation du code, la validation des données et la documentation.

Bibliographie

[1] Vercel, Inc. Next.js documentation. https://nextjs.org/docs.

[2] Prisma Labs. Prisma documentation. https://www.prisma.io/docs.

[3] Meta Platforms, Inc. React documentation. https://react.dev/learn.

[4] Vercel, Inc. Vercel documentation. https://vercel.com/docs.

[5] Neon, Inc. Neon documentation. https://neon.tech/docs.

[6] Microsoft Corporation. Typescript documentation. https://www.typescriptlang.org/docs/.

[7] Zod. Zod documentation. https://zod.dev/.

[8] Tiptap. Tiptap documentation. https://tiptap.dev/docs.

[9] dnd kit. dnd kit documentation. https://dndkit.com/overview.

[10] Cloudinary Ltd. Cloudinary documentation. https://cloudinary.com/documentation.

Glossaire

Terme	Définition
App Router (Next.js)	Système de routage de Next.js basé sur le dossier `app/`, utilisant des *Server Components* par défaut et des *Route Handlers* pour l'API.
API (REST)	Interface permettant à un client (navigateur) d'échanger avec le serveur via des requêtes HTTP (`GET`, `POST`, `PATCH`, `DELETE`) et des réponses structurées (souvent en JSON).
Back-office / Admin	Partie de l'application réservée à l'administration : gestion des produits, catégories, services, rendez-vous, messages, blog et médias.
Cloudinary	Service de stockage et de distribution d'images/vidéos via CDN, avec possibilités d'optimisation et de transformation à la volée.
CDN	*Content Delivery Network* : réseau de serveurs distribué géographiquement qui met en cache et délivre des contenus (ex. images) au plus près des utilisateurs, afin de réduire la latence et d'améliorer les performances.
CRUD	Ensemble d'opérations standards sur des données : *Create, Read, Update, Delete.*
ERD	*Entity Relationship Diagram* : diagramme entité-association représentant les tables, attributs et relations d'une base de données relationnelle.
SEO	*Search Engine Optimization* : ensemble de pratiques visant à améliorer la visibilité d'un site dans les moteurs de recherche (structure des pages, performance, contenus).
SGBD	*Système de Gestion de Base de Données* : logiciel permettant de stocker, organiser et interroger des données (ex. PostgreSQL).
Middleware	Mécanisme exécuté avant le traitement principal d'une requête (ex. ajout d'en-têtes de sécurité, protection d'accès), pouvant s'appliquer à des routes spécifiques.
Neon	Fournisseur de PostgreSQL managé (compatible serverless), souvent utilisé avec des plateformes de déploiement comme Vercel.
ORM (Prisma)	Couche d'abstraction entre la base SQL et le code applicatif, permettant d'écrire des requêtes via un client typé plutôt qu'en SQL brut.
PostgreSQL	SGBD relationnel robuste supportant contraintes, transactions, index, et adapté à des données structurées (rendez-vous, produits, articles).

Terme	Définition
SSR	*Server-Side Rendering* : génération HTML côté serveur pour améliorer SEO et performance perçue.
TypeScript	Sur-ensemble de JavaScript ajoutant un typage statique. Les types sont vérifiés à la compilation, puis effacés à l'exécution.
Vercel	Plateforme de déploiement optimisée pour Next.js, permettant CI/CD, prévisualisations, et hébergement global.
CI/CD	*Continuous Integration / Continuous Delivery* : automatisation des étapes de test, build et déploiement à chaque mise à jour du code.

Annexe A

DD&RS — Développement durable et responsabilité sociale

A.1 Impacts environnementaux et sociétaux (DD&RS)

A.1.1 Impacts liés à l'activité de l'entreprise

L'activité RUHENNA est centrée sur l'artisanat et des prestations événementielles. Plusieurs éléments contribuent à une approche plus responsable :

— **Produits naturels** : RUHENNA met en avant l'usage de produits naturels, avec notamment un approvisionnement auprès d'une entreprise française (**Aroma-Zone**) pour certains composants/matières premières.

— **Artisanat et petites séries** : la production en quantités maîtrisées peut limiter certains gaspillages (sur-stockage), même si elle nécessite une organisation rigoureuse.

A.1.2 Impacts liés au numérique (plateforme web)

La plateforme elle-même induit un impact numérique (hébergement, transfert de données, stockage). Les principaux postes sont :

— **Images** : la galerie et la boutique impliquent beaucoup de médias ; c'est généralement la source principale de trafic et de poids des pages.

— **Hébergement** : déploiement sur Vercel et base de données PostgreSQL managée (Neon) ; l'impact dépend de l'usage réel (trafic, requêtes).

— **Transferts réseau** : consultations des pages, chargements d'images, interactions admin (API).

A.1.3 Impacts liés à l'équipe / au contexte de travail

Dans le contexte d'une petite structure, les impacts de l'équipe sont surtout indirects :

— réduction de la charge administrative grâce à la centralisation (moins d'échanges redondants) ;

— meilleure planification des rendez-vous (optimisation des déplacements et de l'organisation).

A.2 Actions de réduction des impacts

A.2.1 Actions mises en œuvre

Plusieurs choix techniques ont été faits pour limiter l'impact, sans dégrader l'expérience utilisateur :

— **Optimisation des images** : usage de `next/image` et stockage Cloudinary (CDN), pour servir des images adaptées aux tailles d'écran.
— **Architecture simple** : une application Next.js unique (UI + API) réduit la duplication d'infrastructures et la complexité de déploiement.
— **Modèle de données structuré** : PostgreSQL + Prisma limitent les incohérences et réduisent des traitements correctifs coûteux.

Le choix de **Cloudinary** s'explique par la place centrale des images (galerie, produits, blog) : plutôt que de gérer un stockage de fichiers "maison", Cloudinary fournit un CDN et des mécanismes d'optimisation (formats plus légers, compression, tailles adaptées) qui réduisent le poids transféré vers les visiteurs [10]. Couplé à `next/image`, cela aide à limiter les transferts inutiles tout en conservant une bonne qualité visuelle.

A.2.2 Suppression des images côté Cloudinary

Afin d'éviter de stocker inutilement des images non utilisées, la plateforme supprime aussi les médias côté Cloudinary lorsqu'un contenu est supprimé dans l'administration (galerie, bannière, produits, couverture d'article). Cette pratique limite le stockage "orphelin" et garde un système propre [10].

Annexe B

Structure de l'entreprise

Cette annexe décrit la structure organisationnelle de RUHENNA. Il s'agit d'une activité artisanale à taille humaine, centrée autour de sa fondatrice. L'objectif est de clarifier le contexte : la plateforme développée doit être administrable sans équipe technique dédiée, ce qui influence directement les choix d'architecture (simplicité, monorepo Next.js, back-office intégré).

B.1 Organisation générale

— **Direction et production** : pilotées par Sumeyra (vision, fabrication, prestations).
— **Communication** : réseaux sociaux, contenu (photos, articles), interaction avec la clientèle.
— **Relation client** : demandes, devis, suivi des rendez-vous.

B.2 Organigramme simplifié

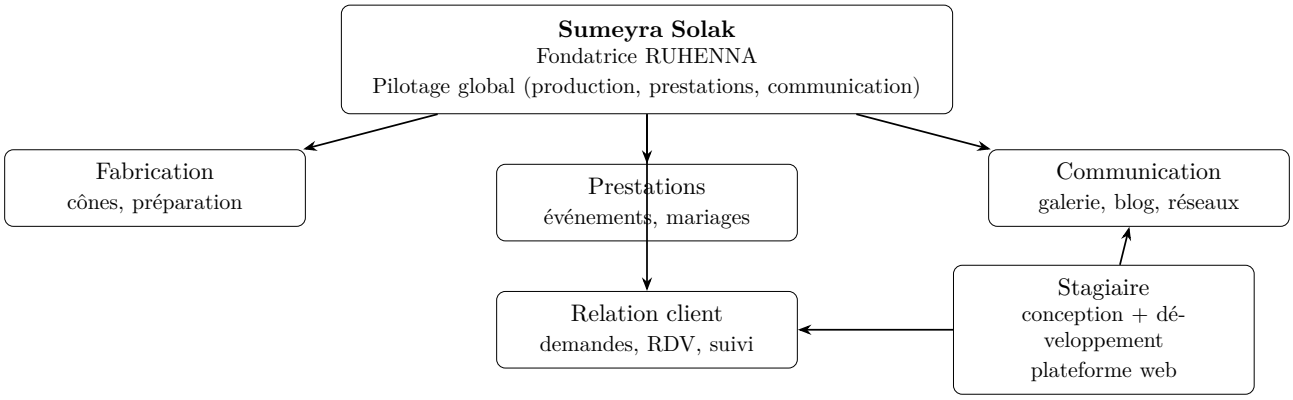

FIGURE B.1 – Organigramme simplifié de RUHENNA et positionnement du stage.

B.3 Conséquences sur le produit

Cette structure implique des exigences fortes :
— **Autonomie** : le back-office doit être intuitif (ajout d'images, tri, publication).
— **Temps limité** : les workflows doivent éviter les tâches répétitives (centralisation messages/rdv).
— **Évolutivité** : possibilité d'ajouter un e-commerce complet plus tard sans réécrire le socle.

Annexe C

Détails techniques

Cette annexe regroupe des éléments techniques complémentaires : inventaire des routes, variables d'environnement utilisées (sans secrets) et explications de configuration.

C.1 Versionnement et stack observée

La stack applicative est décrite dans `package.json`. Les versions exactes peuvent évoluer, mais l'important est la cohérence de l'écosystème (*Next.js/React*, Prisma, PostgreSQL, Tailwind).

Dans ce stage, l'objectif était de rester sur des outils largement documentés et utilisés :

— **Frontend** : React + Next.js [3, 1]
— **Base de données** : PostgreSQL (Neon en production) [5]
— **ORM** : Prisma [2]
— **Styles** : Tailwind CSS
— **Médias** : Cloudinary [10]

C.2 Variables d'environnement

Les secrets (*mots de passe, clés API*) ne doivent pas apparaître dans un document public. La plateforme repose cependant sur les variables suivantes :

— `DATABASE_URL` : chaîne de connexion PostgreSQL (Neon en production).
— `ADMIN_USER`, `ADMIN_PASS` : identifiants Basic Auth pour l'administration.
— `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` : configuration Cloudinary.
— `NEXT_PUBLIC_` : informations publiques de marque et de contact (nom, réseaux, URL du site).

C.3 Structure des dossiers Next.js (public, admin, API)

Le projet utilise l'App Router de Next.js, qui organise l'application par dossiers :

— **Espace public** : `src/app/(main)/...` regroupe les pages accessibles aux visiteurs (`/`, `/boutique`, `/blog`, `/reservation`, etc.). Le layout `src/app/(main)/layout.tsx` applique le header/footer.
— **Espace administration** : `src/app/admin/...` regroupe les pages de gestion (`/admin/produits`, `/admin/services`, `/admin/blog`, etc.), encapsulées par `src/app/admin/layout.tsx`.
— **API** : `src/app/api/.../route.ts` contient les endpoints (publics et admin) appelés depuis les formulaires et l'administration.

Cette structure permet de séparer clairement l'affichage (pages) et les actions (API), tout en restant dans un seul projet.

C.4 Inventaire des principales routes API

Méthode	**Route**	**Implémentation**
GET	`/api/rendez-vous/slots`	`src/app/api/rendez-vous/slots/route.ts` : calcul des créneaux disponibles.
POST	`/api/rendez-vous`	`src/app/api/rendez-vous/route.ts` : création d'une demande de RDV.
POST	`/api/contact`	`src/app/api/contact/route.ts` : enregistrement d'un message.
POST	`/api/upload`	`src/app/api/upload/route.ts` : upload images vers Cloudinary.
GET POST	`/api/admin/products`	`src/app/api/admin/products/route.ts` : liste + création produits (catalogue).
GET PATCH DELETE	`/api/admin/products/[id]`	`src/app/api/admin/products/[id]/route.ts` : lecture + mise à jour + suppression (avec suppression des images associées).
GET POST PATCH	`/api/admin/categories`	`src/app/api/admin/categories/route.ts` : liste + création + réordonnancement des catégories.
PATCH DELETE	`/api/admin/categories/[id]`	`src/app/api/admin/categories/[id]/route.ts` : modification/-suppression (avec réaffectation des produits).
GET POST	`/api/admin/services`	`src/app/api/admin/services/route.ts` : liste + création des prestations.
GET PATCH DELETE	`/api/admin/services/[id]`	`src/app/api/admin/services/[id]/route.ts` : lire/modifier/supprimer une prestation (si aucun RDV en cours).
GET POST DELETE	`/api/admin/services/[id]/availability`	`src/app/api/admin/services/[id]/availability/route.ts` : gérer les règles de disponibilité d'un service.
GET PATCH POST	`/api/admin/rendez-vous`	`src/app/api/admin/rendez-vous/route.ts` : lister/éditer/créer des rendez-vous côté admin.

Méthode	Route	Implémentation
GET	`/api/admin/messages`	`src/app/api/admin/messages/route.ts` : inbox/archives des messages (sans supprimés).
PATCH DELETE	`/api/admin/messages/[id]`	`src/app/api/admin/messages/[id]/route.ts` : marquer lu/archiver + suppression logique.
GET POST	`/api/admin/blog`	`src/app/api/admin/blog/route.ts` : liste + création d'articles.
GET PATCH DELETE	`/api/admin/blog/[id]`	`src/app/api/admin/blog/[id]/route.ts` : lecture + mise à jour + suppression (image de couverture incluse).
GET POST	`/api/admin/galerie`	`src/app/api/admin/galerie/route.ts` : liste + ajout d'images galerie.
PATCH DELETE	`/api/admin/galerie/[id]`	`src/app/api/admin/galerie/[id]/route.ts` : modifier/supprimer (avec suppression Cloudinary).
PATCH	`/api/admin/galerie/reorder`	`src/app/api/admin/galerie/reorder/route.ts` : persister l'ordre (champ `sortOrder`).
GET POST	`/api/admin/hero`	`src/app/api/admin/hero/route.ts` : liste + ajout d'images bannière.
PATCH DELETE	`/api/admin/hero/[id]`	`src/app/api/admin/hero/[id]/route.ts` : activer/désactiver + supprimer (avec suppression Cloudinary).
PATCH	`/api/admin/hero/reorder`	`src/app/api/admin/hero/reorder/route.ts` : persister l'ordre des images "Hero".

C.5 Accès base de données (Prisma Client)

Pour éviter de dupliquer l'initialisation du client Prisma, l'accès DB est centralisé dans un seul module (`src/lib/prisma.ts`). Le principe est de créer une unique instance de client (singleton), puis de la réutiliser dans les pages serveur et les routes API. Cela réduit le risque de multiplier des connexions à PostgreSQL.

C.6 Configuration Cloudinary

Cloudinary est configuré via des variables d'environnement (nom de cloud, clé, secret). L'application enregistre en base uniquement des URLs d'images. Lorsqu'un contenu est supprimé dans l'administration, une suppression côté Cloudinary est déclenchée afin d'éviter de conserver des médias orphelins (stockage inutile) [10].

C.7 Optimisation des images côté Next.js

Pour permettre à `next/image` d'afficher des images hébergées sur Cloudinary, l'application autorise explicitement le domaine `res.cloudinary.com` dans la configuration Next.js (*remote patterns*) [1].

C.8 Dépôt GitHub (lien du code)

`https://github.com/<organisation-ou-utilisateur>/<nom-du-repo>`